■ PYTHON PROGRAMMING

Complete SEE Exam Notes

Chapter 4 | CDC Curriculum 2083

■ Class	Class 10 (SEE)
■ Subject	Computer Science
■ Chapter	Chapter 4: Programming in Python
■ Curriculum	CDC Nepal 2083
■ Focus	SEE Board Examination
■ Language	Simple English

■ TOPICS COVERED IN THIS CHAPTER

- ✔ Introduction to Python
- ✔ Variables, Data Types & Operators
- ✔ Input / Output Statements
- ✔ Conditional Statements (if/elif/else)
- ✔ Looping Statements (for / while)
- ✔ Functions (Built-in & User-defined)
- ✔ Lists, Tuples & Dictionaries
- ✔ String Operations
- ✔ Programs with Explanations
- ✔ SEE Important Questions & MCQs

These notes cover ALL concepts of Chapter 4 (Programming in Python) as per CDC Curriculum 2083. Students can study directly from these notes without reading the textbook. All SEE Board Exam patterns are followed.

SECTION : TABLE OF CONTENTS

#	Topic	Page
1	Introduction to Python	3
2	Key Concepts	4
3	Important Definitions	6
4	Important Points for SEE Exam	7
5	Rules, Syntax & Commands	8
6	Difference Tables	10
7	Programs with Line-by-Line Explanation	13
8	Common Exam Mistakes	17
9	SEE Important Questions & Answers	18
10	Multiple Choice Questions (MCQs)	22
11	True/False Questions	25
12	Fill in the Blanks	26
13	One-Mark Questions	27
14	Quick Revision Sheet	28
15	Exam Tips & Memory Tricks	30

SECTION 1: INTRODUCTION TO PYTHON

What is Python?

Definition: Python is a high-level, general-purpose, interpreted programming language that is easy to learn and use. It was created by **Guido van Rossum** in **1991**.

Simple Explanation: Python is a computer language that lets you tell the computer what to do using simple English-like commands. You don't need to worry about complex rules — Python does the hard work for you.

Real-Life Analogy: Just like we give instructions in simple Nepali or English to a friend, Python lets us give instructions to a computer in a simple, readable way.

Features of Python

Feature	Explanation
Easy to Learn	Python syntax is simple and similar to English. Beginners can learn it quickly.
Free & Open Source	Python is available for free. Anyone can download and use it.
Interpreted Language	Python runs code line by line. No need to compile before running.
Platform Independent	Python programs can run on Windows, Mac, Linux without any changes.
Object-Oriented	Python supports Object-Oriented Programming (OOP) concepts.
Large Library Support	Python has thousands of built-in libraries for science, web, AI, etc.
Dynamically Typed	You don't need to declare data types. Python finds the type automatically.
Case-Sensitive	Python treats uppercase and lowercase letters differently. 'Name' ≠ 'name'.

Applications of Python

- Web Development (Django, Flask frameworks)
- Artificial Intelligence (AI) and Machine Learning (ML)
- Data Science and Data Analysis
- Game Development (using Pygame library)

- Scientific and Numeric Computing (NumPy, SciPy)
- Desktop GUI Applications (Tkinter)
- Automation and Scripting
- Cybersecurity and Ethical Hacking

Python Versions

Version	Description
Python 2.x	Older version. No longer supported since 2020.
Python 3.x	Current version. Used for all modern development. (Class 10 uses Python 3)

SECTION 2: KEY CONCEPTS

2.1 Variables & Data Types

A **variable** is a named memory location used to store data. In Python, you don't need to declare the type — Python automatically detects it.

Rules for Naming Variables:

- Variable name must start with a letter (a-z, A-Z) or underscore (_).
- Variable name cannot start with a number.
- Variable names are case-sensitive (age ≠ Age ≠ AGE).
- No spaces allowed. Use underscore instead (my_name, not my name).
- Cannot use Python keywords (if, for, while, etc.) as variable names.
- Can contain letters, numbers, and underscores only.

Data Types in Python:

Data Type	Keyword	Example	Description
Integer	int	x = 10	Whole numbers (positive or negative)
Float	float	y = 3.14	Decimal numbers
String	str	name = "Ram"	Text enclosed in quotes
Boolean	bool	flag = True	True or False only
List	list	marks = [45, 60, 75]	Ordered, changeable collection
Tuple	tuple	t = (1, 2, 3)	Ordered, unchangeable collection
Dictionary	dict	d = {"name":"Ram"}	Key-value pairs
NoneType	None	x = None	Represents no value / empty

```
# Variable Assignment Examples
name = 'Sita'          # String
age  = 16              # Integer
marks = 89.5           # Float
passed = True          # Boolean
print(type(name))      # Output: <class 'str'>
print(type(age))       # Output: <class 'int'>
```

2.2 Operators

Operators are symbols that perform operations on variables and values.

Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	5 + 3	8
-	Subtraction	10 - 4	6
*	Multiplication	4 * 3	12
/	Division	10 / 4	2.5
//	Floor Division	10 // 3	3
%	Modulus (Remainder)	10 % 3	1
**	Exponentiation (Power)	2 ** 3	8

Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	8 > 3	True
<	Less than	3 < 8	True
>=	Greater or equal	5 >= 5	True
<=	Less or equal	3 <= 5	True

Logical Operators

Operator	Meaning	Example	Result
and	True if BOTH conditions are True	5>3 and 4>2	True
or	True if AT LEAST ONE condition is True	5>3 or 1>9	True
not	Reverses the result	not(5>3)	False

Assignment Operators

Operator	Meaning	Example	Result
=	Assign value	x = 10	x becomes 10
+=	Add and assign	x += 5	x = x + 5

Operator	Meaning	Example	Result
-=	Subtract and assign	x -= 3	x = x - 3
*=	Multiply and assign	x *= 2	x = x * 2
/=	Divide and assign	x /= 4	x = x / 4
//=	Floor div & assign	x //= 3	x = x // 3
%=	Modulus and assign	x %= 3	x = x % 3

2.3 Input and Output

Python uses **print()** for output and **input()** for taking input from the user.

Function	Purpose	Example	Result/Note
print()	Displays output to the screen	print("Hello, World!")	Hello, World!
input()	Takes input from the user (always returns string)	name = input("Enter name: ")	Waits for user input
int()	Converts string to integer	x = int(input('Enter: '))	Converts '5' to 5
float()	Converts string to float	x = float(input('Enter: '))	Converts '3.5' to 3.5
str()	Converts value to string	str(100)	'100'

```
# Basic Input/Output Example
name = input('Enter your name: ')     # Takes text input
age  = int(input('Enter your age: '))  # Takes integer input
print('Hello,', name)                  # Output: Hello, [name]
print('Your age is:', age)             # Output: Your age is: [age]

# print() formatting options
print('Sum =', 5 + 3)                 # Output: Sum = 8
print('Pi =', 3.14159)
print('A', 'B', 'C', sep='-')         # Output: A-B-C
print('Hello', end=' ')               # No newline at end
print('World')                        # Output: Hello World
```

2.4 Conditional Statements (if / elif / else)

Conditional statements allow a program to make decisions and execute different code based on conditions.

Statement	Purpose
if	Executes a block if the condition is True
if...else	Executes one block if True, another if False
if...elif...else	Checks multiple conditions one by one
Nested if	An if statement inside another if statement

Syntax:

```
# if Statement
if condition:
    statement(s)

# if...else Statement
if condition:
    statement(s)   # runs when True
else:
    statement(s)   # runs when False

# if...elif...else Statement
if condition1:
    statement(s)
elif condition2:
    statement(s)
elif condition3:
    statement(s)
else:
    statement(s)
```

IMPORTANT: Python uses indentation (spaces/tab) to define blocks of code. Always use consistent indentation (4 spaces recommended). Forgetting indentation is the #1 student mistake!

2.5 Looping Statements (for / while)

Loops allow a set of statements to be executed repeatedly. Python has two main types of loops: **for** loop and **while** loop.

for Loop Syntax:

```
for variable in sequence:
    statement(s)

# Using range()
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):       # 1, 2, 3, 4, 5
    print(i)

for i in range(1, 11, 2):   # 1, 3, 5, 7, 9 (step=2)
    print(i)
```

while Loop Syntax:

```
while condition:
    statement(s)
    (update condition to avoid infinite loop)

# Example:
i = 1
while i <= 5:
    print(i)
    i += 1          # i = i + 1  (MUST update!)
```

Loop Control Statements:

Statement	What it does	Use Case
break	Exits the loop immediately	Stop the loop when a condition is met
continue	Skips the current iteration and goes to next	Skip printing even numbers
pass	Does nothing; used as a placeholder	Empty loop body

```
# break example
for i in range(1, 11):
    if i == 5:
        break        # Loop stops when i becomes 5
    print(i)
# Output: 1 2 3 4

# continue example
for i in range(1, 6):
    if i == 3:
        continue     # Skips 3
    print(i)
# Output: 1 2 4 5
```

2.6 Functions

A **function** is a block of reusable code that performs a specific task. Functions help avoid repetition and make code organized.

Types of Functions:

Type	Description	Example
Built-in Functions	Already available in Python. No need to define.	print(), input(), len(), type(), range(), int(), float(), str(), abs(), max(), min(), sum()
User-defined Functions	Created by the programmer using def keyword.	def greet(): print('Hello')
Lambda Functions	Small anonymous (nameless) functions.	square = lambda x: x * x

Defining and Calling a Function:

```
# Syntax to define a function
def function_name(parameter1, parameter2):
    statement(s)
    return value    # optional

# Calling a function
function_name(argument1, argument2)

# Example 1: Function with no parameters
def greet():
    print('Hello, Welcome to Python!')

greet()    # Calling the function
# Output: Hello, Welcome to Python!

# Example 2: Function with parameters & return
def add(a, b):
    result = a + b
    return result

total = add(5, 3)
print('Sum =', total)    # Output: Sum = 8
```

Parameter vs Argument:

- **Parameter:** Variable listed in the function **definition**. Example: def add(**a, b**)
- **Argument:** Actual value passed when **calling** the function. Example: add(**5, 3**)

Variable Scope (Local vs Global):

Type	Where defined	Where accessible	Example
Local Variable	Defined inside a function	Only accessible within that function	def f(): x = 10
Global Variable	Defined outside all functions	Accessible throughout the program	x = 10 def f(): print(x)

```
x = 100           # Global variable

def show():
    y = 50        # Local variable
    print(x)      # Can access global variable
    print(y)      # Can access local variable

show()
# print(y)        # ERROR! y is not accessible outside function
```

2.7 Lists, Tuples & Dictionaries

Lists:

A list is an **ordered, mutable (changeable)** collection of items. Items can be of different data types. Lists use **square brackets [ ]**.

```
marks = [45, 78, 90, 55, 66]   # List of integers
fruits = ['apple', 'banana', 'mango']

# Accessing elements (index starts from 0)
print(marks[0])      # Output: 45 (first element)
print(marks[-1])     # Output: 66 (last element)

# Changing elements
marks[1] = 80        # marks becomes [45, 80, 90, 55, 66]

# Common List Methods
marks.append(95)     # Adds 95 at the end
marks.remove(55)     # Removes first occurrence of 55
marks.sort()         # Sorts in ascending order
marks.reverse()      # Reverses the list
print(len(marks))    # Length of list
print(marks.index(90)) # Index of 90
```

Tuples:

A tuple is an **ordered, immutable (cannot be changed)** collection. Tuples use **parentheses ( )**.

```
colors = ('red', 'green', 'blue')   # Tuple
print(colors[0])     # Output: red
print(len(colors))   # Output: 3
# colors[0] = 'yellow'  # ERROR! Tuples cannot be changed

# Tuple methods
print(colors.count('red'))   # Counts occurrences
print(colors.index('green')) # Returns index
```

Dictionaries:

A dictionary stores data as **key-value pairs**. It uses **curly braces { }**. Keys must be unique.

```
student = {'name': 'Ram', 'age': 16, 'marks': 85}

# Accessing values
print(student['name'])     # Output: Ram
print(student.get('age'))  # Output: 16

# Adding / Updating
student['grade'] = 'A'     # Add new key-value
student['age'] = 17        # Update existing value

# Removing
del student['marks']       # Delete a key

# Dictionary methods
print(student.keys())      # All keys
print(student.values())    # All values
print(student.items())     # All key-value pairs
```

String Operations:

Strings in Python are sequences of characters enclosed in quotes.

```
s = 'Hello, Nepal!'
print(s.upper())        # HELLO, NEPAL!
print(s.lower())        # hello, nepal!
print(s.replace('Nepal','World'))  # Hello, World!
print(s.split(','))     # ['Hello', ' Nepal!']
print(s[0:5])           # Hello   (slicing)
print(len(s))           # 13
print(s.strip())        # Removes leading/trailing spaces
print(s.count('l'))     # 3  (counts 'l')
print(s.find('Nepal'))  # 7  (starting index)

# String concatenation
first = 'Good'
second = 'Morning'
print(first + ' ' + second)  # Good Morning

# String repetition
print('Ha' * 3)   # HaHaHa
```

SECTION 3: IMPORTANT DEFINITIONS

Python: A high-level, general-purpose, interpreted programming language created by Guido van Rossum in 1991.

Variable: A named memory location used to store data that can change during program execution.

Data Type: Classification of data that tells Python what kind of value a variable holds (int, float, str, bool, etc.).

Integer (int): A data type representing whole numbers without decimal points. Example: 5, -3, 100.

Float: A data type representing numbers with decimal points. Example: 3.14, -2.5.

String (str): A sequence of characters enclosed in single or double quotes. Example: 'Hello', "Nepal".

Boolean: A data type with only two values: True or False.

Operator: A symbol that performs an operation on one or more operands. Example: +, -, *, /, %, **.

print(): A built-in function used to display output on the screen.

input(): A built-in function used to take input from the user as a string.

if Statement: A conditional statement that executes a block of code only if the given condition is True.

else Clause: An optional part of if statement that executes when the condition is False.

elif Clause: Short for 'else if'; checks another condition if the previous one is False.

for Loop: A loop that repeats a block of code for each item in a sequence or range.

while Loop: A loop that repeats a block of code as long as the given condition remains True.

range(): A built-in function that generates a sequence of numbers. Example: range(1,6) gives 1,2,3,4,5.

break: A loop control statement that immediately exits (terminates) the current loop.

continue: A loop control statement that skips the current iteration and moves to the next one.

Function: A named, reusable block of code that performs a specific task.

def: A keyword used to define (create) a user-defined function in Python.

Parameter: A variable listed in the function definition that receives values from the caller.

Argument: The actual value passed to a function when calling it.

return: A keyword used in a function to send back a value to the caller.

Built-in Function: Functions already available in Python that do not need to be defined. Example: print(), len().

User-defined Function: Functions created by the programmer using the def keyword.

Local Variable: A variable defined inside a function, accessible only within that function.

Global Variable: A variable defined outside all functions, accessible throughout the entire program.

List: An ordered, mutable collection of items enclosed in square brackets [ ].

Tuple: An ordered, immutable collection of items enclosed in parentheses ( ).

Dictionary: A collection of key-value pairs enclosed in curly braces { }.

String: A sequence of characters used to represent text.

Index: The position of an element in a sequence (starts from 0 in Python).

Slicing: Extracting a part of a string or list using [start:end] notation.

Indentation: Spaces or tabs at the beginning of a line that define a code block in Python.

Syntax Error: An error caused by incorrect Python syntax (grammar rules).

Runtime Error: An error that occurs while the program is running (e.g., dividing by zero).

Logical Error: An error where the program runs without crashing but produces wrong output.

Comment: Text in the code that Python ignores, used to explain the code. Starts with #.

Module: A single Python file (.py) that contains functions, variables, and classes.

Library: A collection of modules used to provide extra functionality.

SECTION 4: IMPORTANT POINTS FOR SEE EXAM

■ IMPORTANT FOR SEE EXAM

- Python was created by **Guido van Rossum** in **1991**.
- Python is an **interpreted, high-level, general-purpose** programming language.
- Python is **case-sensitive**: 'name' and 'Name' are different variables.
- Python uses **indentation** (spaces/tabs) to define blocks — NOT curly braces like C.
- Comments in Python start with #. They are ignored by the interpreter.
- The **print()** function is used for output; **input()** is used for input.
- **input()** always returns a **string**. Use int() or float() to convert.
- **range(start, stop, step)**: stop value is **not included** in the output.
- List index **starts from 0**. Negative index counts from the end (-1 = last element).
- List is **mutable** (can be changed); Tuple is **immutable** (cannot be changed).
- Dictionary uses **key-value pairs**. Keys must be **unique**.
- A **parameter** is in function definition; an **argument** is in function call.
- **Local variables** exist only inside the function; **global variables** exist everywhere.
- **break** exits the loop; **continue** skips to the next iteration.
- **Syntax Error** = wrong grammar; **Runtime Error** = crash during execution; **Logical Error** = wrong output.
- Python's **** operator** is used for power/exponentiation (2**3 = 8).
- Python's **// operator** is floor division (returns integer quotient).
- Python's **% operator** returns remainder (modulus).
- String slicing: **s[start:end]** — includes start, excludes end.
- Use **def** keyword to define a function; **return** to return a value.
- The **len()** function returns the number of items in a list/string/tuple.
- **append()** adds element at end of list; **remove()** removes by value; **pop()** removes by index.
- An **infinite loop** occurs when the while loop condition never becomes False.
- Python file extension is **.py**
- To use type conversion: **int(), float(), str(), bool()**

SECTION 5: RULES AND SYNTAX

print() Syntax

```
print(value)              # Single value
print(value1, value2)     # Multiple values
print('Text', variable)   # Text + variable
print(value, end='\n')   # Default end is newline
print(a, b, sep='-')      # Custom separator
```

input() Syntax

```
variable = input('prompt message: ')
# Returns a STRING
name = input('Enter name: ')
age  = int(input('Enter age: '))    # Convert to int
gpa  = float(input('Enter GPA: '))  # Convert to float
```

if / elif / else

```
if condition:
    block1
elif condition2:
    block2
else:
    block3
```

for Loop

```
for variable in range(n):      # 0 to n-1
    statement

for variable in range(a, b):   # a to b-1
    statement

for variable in range(a, b, s): # a to b-1, step s
    statement

for item in list_name:         # Loop through list
    statement
```

while Loop

```
while condition:
    statement(s)
    update_variable          # Prevent infinite loop
```

Function Definition & Call

```
# Definition
def function_name(param1, param2):
    statement(s)
    return value

# Calling
result = function_name(arg1, arg2)
```

List Operations

```
my_list = [1, 2, 3, 4, 5]
my_list.append(6)         # Add to end
my_list.insert(0, 0)      # Insert at index 0
my_list.remove(3)         # Remove by value
my_list.pop(2)            # Remove by index
my_list.sort()            # Sort ascending
my_list.reverse()         # Reverse
len(my_list)              # Length
my_list[1:4]              # Slicing index 1,2,3
```

Dictionary Operations

```
d = {'key1': val1, 'key2': val2}
d['key1']                  # Access value
d.get('key1')              # Safe access
d['new_key'] = val         # Add/update
del d['key1']              # Delete key
d.keys()                   # All keys
d.values()                 # All values
d.items()                  # All pairs
'key1' in d                # Check if key exists
```

SECTION 6: DIFFERENCE TABLES

Built-in Function vs User-defined Function

Aspect	Built-in Function	User-defined Function
Definition	Already defined in Python	Defined by the programmer
Keyword needed	Not required	Uses def keyword
Examples	print(), input(), len(), type()	def greet():, def add(a,b):
Availability	Always available	Available only after defining
Modification	Cannot be modified	Can be modified as needed
Purpose	General purpose	Specific to programmer's need

Parameter vs Argument

Aspect	Parameter	Argument
Location	In function definition	In function call
What it is	A variable name placeholder	Actual value passed to function
Example	def add(a, b): ← a,b are parameters	add(5, 3) ← 5,3 are arguments
Type	Variable	Value / expression

Local Variable vs Global Variable

Aspect	Local Variable	Global Variable
Where defined	Inside a function	Outside all functions
Accessibility	Only within that function	Entire program
Lifetime	Created when function called; destroyed when function ends	Exists throughout program
Example	def f(): x=10	x = 10 def f(): print(x)

List vs Tuple vs Dictionary

Aspect	List	Tuple	Dictionary
Brackets	[ ]	( )	{ }
Mutable?	Yes (can change)	No (cannot change)	Yes (can change)
Ordered?	Yes	Yes	No (Python 3.7+ maintains insertion order)
Duplicate values?	Yes	Yes	Keys must be unique
Use case	General-purpose data	Fixed/constant data	Key-value mapping
Example	[1,2,3]	(1,2,3)	{'name':'Ram'}

for Loop vs while Loop

Aspect	for Loop	while Loop
Used when	Number of iterations is known	Number of iterations is unknown
Condition	Based on sequence / range	Based on a condition
Risk of infinite loop	Rare	High if condition never becomes False
Syntax	for i in range(n):	while condition:
Example use	Print 1 to 10	Keep asking until correct input

Syntax Error vs Runtime Error vs Logical Error

Aspect	Syntax Error	Runtime Error	Logical Error
When occurs	Before program runs	During execution	Program runs but gives wrong output
Detected by	Python interpreter	Python interpreter	Programmer (manual check)
Example	Missing colon:	Dividing by zero	Wrong formula used
Also called	Compile-time error	Exception	Semantic error
Fixes	Fix grammar/syntax	Handle exceptions	Debug logic

Module vs Package vs Library

Aspect	Module	Package	Library
Definition	A single .py file with code	A folder of modules with __init__.py	Collection of packages/modules
Size	Smallest unit	Medium	Largest
Example	math.py, random.py	numpy (folder)	NumPy, Pandas, Django
How to use	import math	import numpy	import pandas

String vs List

Aspect	String	List
Definition	Sequence of characters	Ordered collection of any data
Mutable?	No (immutable)	Yes (mutable)
Brackets	' ' or " "	[ ]
Elements	Only characters	Any data type
Example	'Nepal'	['a', 1, True]

SECTION 7: IMPORTANT COMMANDS AND FUNCTIONS

Function/Command	Purpose	Syntax	Example
print()	Display output	print(value)	print("Hello")
input()	Take user input (returns string)	input('msg')	x = input('Enter: ')
int()	Convert to integer	int(value)	int('5') → 5
float()	Convert to float	float(value)	float('3.5') → 3.5
str()	Convert to string	str(value)	str(10) → '10'
bool()	Convert to boolean	bool(value)	bool(0) → False
type()	Return data type	type(value)	type(5) →
len()	Length of string/list/tuple	len(obj)	len('Nepal') → 5
range()	Generate number sequence	range(start,stop,step)	range(1,6) → 1,2,3,4,5
abs()	Absolute value	abs(x)	abs(-5) → 5
max()	Maximum value	max(a,b,...)	max(3,7,2) → 7
min()	Minimum value	min(a,b,...)	min(3,7,2) → 2
sum()	Sum of elements	sum(list)	sum([1,2,3]) → 6
round()	Round to n decimal places	round(x,n)	round(3.14159,2) → 3.14
sorted()	Return sorted list	sorted(list)	sorted([3,1,2]) → [1,2,3]
reversed()	Return reversed iterator	reversed(list)	list(reversed([1,2,3]))
list.append()	Add item to end of list	list.append(item)	a.append(5)
list.remove()	Remove item by value	list.remove(item)	a.remove(3)
list.pop()	Remove item by index	list.pop(index)	a.pop(0)
list.sort()	Sort list in place	list.sort()	a.sort()
list.reverse()	Reverse list in place	list.reverse()	a.reverse()
list.insert()	Insert at specific index	list.insert(i,val)	a.insert(1,'x')
list.index()	Find index of value	list.index(val)	a.index(5)
list.count()	Count occurrences	list.count(val)	a.count(3)

Function/Command	Purpose	Syntax	Example
str.upper()	Convert to uppercase	str.upper()	'hello'.upper() → HELLO
str.lower()	Convert to lowercase	str.lower()	'HELLO'.lower() → hello
str.strip()	Remove whitespace	str.strip()	' hi '.strip() → 'hi'
str.replace()	Replace substring	str.replace(old,new)	'a b'.replace(' ','-')
str.split()	Split string into list	str.split(sep)	'a,b'.split(',') → ['a','b']
str.find()	Find substring index	str.find(sub)	'Nepal'.find('e') → 1
str.count()	Count occurrences	str.count(sub)	'banana'.count('a') → 3
dict.keys()	Get all keys	dict.keys()	d.keys()
dict.values()	Get all values	dict.values()	d.values()
dict.items()	Get all key-value pairs	dict.items()	d.items()
dict.get()	Safe access by key	dict.get(key)	d.get('name')

SECTION 8: PROGRAMS WITH LINE-BY-LINE EXPLANATION

Program 1: Print Hello World

Program:

```
print("Hello, World!")     # Line 1
```

Output: Hello, World!

Line-by-Line Explanation:

- Line 1: print() is a built-in function that displays text. The text inside quotes is shown on screen.

Program 2: Input & Display Name

Program:

```
name = input('Enter your name: ')   # Line 1
print('Hello,', name)                # Line 2
```

Output: Enter your name: Ram
Hello, Ram

Line-by-Line Explanation:

- Line 1: input() waits for user to type. The text is stored in variable 'name' as a string.
- Line 2: print() shows 'Hello,' followed by the value in 'name'.

Program 3: Add Two Numbers

Program:

```
a = int(input('Enter first number: '))    # Line 1
b = int(input('Enter second number: '))   # Line 2
sum = a + b                               # Line 3
print('Sum =', sum)                       # Line 4
```

Output: Enter first number: 5
Enter second number: 3
Sum = 8

Line-by-Line Explanation:

- Line 1: Takes input and converts it to integer using int().
- Line 2: Takes second integer input.
- Line 3: Adds a and b and stores result in 'sum'.
- Line 4: Displays the result.

Program 4: Check Even or Odd

Program:

```
n = int(input('Enter a number: '))    # Line 1
if n % 2 == 0:                         # Line 2
    print(n, 'is Even')                # Line 3
else:                                  # Line 4
    print(n, 'is Odd')                 # Line 5
```

Output: Enter a number: 4
4 is Even

Line-by-Line Explanation:

- Line 1: Takes an integer input from user.
- Line 2: Checks if n divided by 2 gives remainder 0 (i.e., even).
- Line 3: If condition is True, prints 'is Even'.
- Line 4: else runs when the condition is False.
- Line 5: Prints 'is Odd' when n is not divisible by 2.

Program 5: Find Largest of Three Numbers

Program:

```
a = int(input('Enter a: '))    # Line 1
b = int(input('Enter b: '))    # Line 2
c = int(input('Enter c: '))    # Line 3
if a >= b and a >= c:          # Line 4
    print(a, 'is largest')     # Line 5
elif b >= a and b >= c:        # Line 6
    print(b, 'is largest')     # Line 7
else:                          # Line 8
    print(c, 'is largest')     # Line 9
```

Output: Enter a: 5
Enter b: 9
Enter c: 3
9 is largest

Line-by-Line Explanation:

- Lines 1-3: Take three integer inputs.
- Line 4: Check if 'a' is greater than or equal to both 'b' and 'c'.
- Line 5: If True, print 'a' as largest.
- Line 6: elif checks if 'b' is largest.
- Line 7: Print 'b' as largest.
- Lines 8-9: If neither is true, 'c' must be largest.

Program 6: Print 1 to 10 Using for Loop

Program:

```
for i in range(1, 11):     # Line 1
    print(i)               # Line 2
```

Output: 1
2
3
4
5
6
7
8
9
10

Line-by-Line Explanation:

- Line 1: range(1, 11) generates numbers from 1 to 10. The loop runs for each value.
- Line 2: Prints the current value of i in each iteration.

Program 7: Print Multiplication Table

Program:

```
n = int(input('Enter number: '))   # Line 1
for i in range(1, 11):             # Line 2
    print(n, 'x', i, '=', n*i)    # Line 3
```

Output: Enter number: 5
5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50

Line-by-Line Explanation:

- Line 1: Takes number from user.
- Line 2: Loop runs from i=1 to i=10.
- Line 3: Prints n × i = result for each iteration.

Program 8: Sum of Numbers Using while Loop

Program:

```
total = 0              # Line 1
i = 1                  # Line 2
while i <= 10:         # Line 3
    total += i         # Line 4
    i += 1             # Line 5
print('Sum =', total)  # Line 6
```

Output: Sum = 55

Line-by-Line Explanation:

- Line 1: Initialize total to 0.
- Line 2: Initialize counter i to 1.
- Line 3: Loop continues while i is ≤ 10.
- Line 4: Add i to total (total = total + i).
- Line 5: Increment i by 1 (IMPORTANT to avoid infinite loop).
- Line 6: Print the final sum after loop ends.

Program 9: User-defined Function to Find Factorial

Program:

```
def factorial(n):          # Line 1
    result = 1             # Line 2
    for i in range(1, n+1):  # Line 3
        result *= i        # Line 4
    return result          # Line 5

num = int(input('Enter: '))  # Line 6
print('Factorial =', factorial(num))  # Line 7
```

Output: Enter: 5
Factorial = 120

Line-by-Line Explanation:

- Line 1: Define a function named 'factorial' with parameter n.
- Line 2: Initialize result to 1.
- Line 3: Loop from 1 to n (inclusive).
- Line 4: Multiply result by each i.
- Line 5: Return the computed factorial.
- Line 6: Take input from user.
- Line 7: Call function and print result.

Program 10: Check Prime Number

Program:

```
n = int(input('Enter number: '))   # Line 1
is_prime = True                     # Line 2
if n < 2:                           # Line 3
    is_prime = False
for i in range(2, n):               # Line 4
    if n % i == 0:                  # Line 5
        is_prime = False            # Line 6
        break                       # Line 7
if is_prime:                        # Line 8
    print(n, 'is Prime')
else:
    print(n, 'is Not Prime')
```

Output: Enter number: 7
7 is Prime

Line-by-Line Explanation:

- Line 1: Take integer input.
- Line 2: Assume the number is prime initially.
- Line 3: Numbers less than 2 are not prime.
- Line 4: Check divisibility from 2 up to n-1.
- Line 5: If n is divisible by any i, it's not prime.
- Line 6: Set is_prime to False.
- Line 7: No need to check further — break out.
- Line 8: Print result based on is_prime flag.

Program 11: List Operations

Program:

```
students = ['Ram', 'Sita', 'Hari']   # Line 1
students.append('Gita')               # Line 2
print('List:', students)              # Line 3
print('Length:', len(students))       # Line 4
print('First:', students[0])          # Line 5
print('Last:', students[-1])          # Line 6
students.sort()                       # Line 7
print('Sorted:', students)            # Line 8
```

Output: List: ['Ram', 'Sita', 'Hari', 'Gita']
Length: 4
First: Ram
Last: Gita
Sorted: ['Gita', 'Hari', 'Ram', 'Sita']

Line-by-Line Explanation:

- Line 1: Create a list of student names.
- Line 2: Add 'Gita' at the end using append().
- Line 3: Print the full list.
- Line 4: Print number of elements using len().
- Line 5: Access first element (index 0).
- Line 6: Access last element (index -1).
- Line 7: Sort the list alphabetically.
- Line 8: Print the sorted list.

Program 12: Dictionary Example – Student Record

Program:

```
student = {                          # Line 1
    'name': 'Prakash',
    'age': 16,
    'marks': 85
}
print('Name:', student['name'])      # Line 2
print('Age:', student['age'])        # Line 3
student['grade'] = 'A'               # Line 4
print('Updated:', student)           # Line 5
```

Output: Name: Prakash
Age: 16
Updated: {'name': 'Prakash', 'age': 16, 'marks': 85, 'grade': 'A'}

Line-by-Line Explanation:

- Line 1: Create a dictionary with three key-value pairs.
- Line 2: Access value using key 'name'.
- Line 3: Access value using key 'age'.
- Line 4: Add new key 'grade' with value 'A'.
- Line 5: Print the updated dictionary.

SECTION 8: COMMON EXAM MISTAKES

■ COMMON EXAM MISTAKES

- NOT converting input() to int/float before arithmetic (input() always returns a string).
- Forgetting the colon (:) after if, else, elif, for, while, def.
- Using wrong indentation (mixing tabs and spaces in the same block).
- Confusing = (assignment) with == (comparison). Example: if x = 5: is WRONG; use if x == 5:
- Off-by-one error in range(): range(1,10) gives 1 to 9, NOT 1 to 10.
- Forgetting to increment counter in while loop, causing infinite loop.
- Trying to change a tuple (tuples are immutable — they cannot be modified).
- Using the wrong quotes: mixing ' and " in the same string. Example: 'Hello" is wrong.
- Accessing list index that doesn't exist (IndexError).
- Confusing append() (adds item) with extend() (adds all items from another list).
- Writing print without () — in Python 3, print is a FUNCTION, so () is required.
- Calling a function before defining it — the def block must come before the call.
- Forgetting return in a function when a value is needed outside.
- Writing elif before if — elif cannot exist without an if first.
- Confusing parameter (in definition) with argument (in call).
- Dividing by zero without checking — causes ZeroDivisionError at runtime.
- Using list methods on strings or vice versa (e.g., append() doesn't work on strings).
- Not using break in a loop when searching, causing unnecessary iterations.
- Forgetting that negative indexing in Python starts from -1 (last element), not 0.
- Writing 'and', 'or', 'not' as '&&', '||', '!' — Python does NOT use these symbols.

SECTION 9: SEE IMPORTANT QUESTIONS & ANSWERS

A. VERY SHORT ANSWER QUESTIONS (1–2 marks)

Q1. What is Python?

Ans: Python is a high-level, general-purpose, interpreted programming language created by Guido van Rossum in 1991.

Q2. Who created Python?

Ans: Python was created by Guido van Rossum.

Q3. What is a variable in Python?

Ans: A variable is a named memory location used to store data that can change during program execution.

Q4. What is the purpose of print() function?

Ans: The print() function is used to display output on the screen.

Q5. What is the purpose of input() function?

Ans: The input() function is used to take input from the user. It always returns a string.

Q6. Name four data types in Python.

Ans: int, float, str, bool (also list, tuple, dict, NoneType).

Q7. What is indentation in Python?

Ans: Indentation refers to spaces or tabs at the beginning of a line that are used to define blocks of code in Python.

Q8. What does range(1, 6) produce?

Ans: range(1, 6) produces the sequence: 1, 2, 3, 4, 5 (stop value 6 is not included).

Q9. What is the difference between = and ==?

Ans: '=' is the assignment operator used to assign values. '==' is the comparison operator used to check equality.

Q10. What is an infinite loop?

Ans: An infinite loop is a loop that never ends because its condition always remains True.

Q11. What is the use of break statement?

Ans: The break statement is used to immediately exit (terminate) a loop.

Q12. What is the use of continue statement?

Ans: The continue statement is used to skip the current iteration of a loop and move to the next one.

Q13. What is a function in Python?

Ans: A function is a named, reusable block of code that performs a specific task.

Q14. What keyword is used to define a function?

Ans: The def keyword is used to define a function in Python.

Q15. What is the difference between a list and a tuple?

Ans: A list is mutable (can be changed) and uses [ ]. A tuple is immutable (cannot be changed) and uses ( ).

Q16. What is a dictionary in Python?

Ans: A dictionary is a collection of key-value pairs enclosed in curly braces { }.

Q17. What does len() function do?

Ans: The len() function returns the number of elements in a string, list, or tuple.

Q18. What is a local variable?

Ans: A local variable is a variable defined inside a function, accessible only within that function.

Q19. What is a global variable?

Ans: A global variable is a variable defined outside all functions, accessible throughout the entire program.

Q20. What is a syntax error?

Ans: A syntax error is an error caused by incorrect Python syntax (grammar) that prevents the program from running.

B. SHORT ANSWER QUESTIONS (3–5 marks)

Q1. Explain the features of Python with examples.

Ans: Python features include: (i) Easy to learn — simple English-like syntax. (ii) Interpreted — runs line by line without compilation. (iii) Platform independent — runs on Windows, Mac, Linux. (iv) Free & Open Source — available for free download. (v) Dynamically typed — data types detected automatically. (vi) Case-sensitive — 'name' ≠ 'Name'. (vii) Large library support — thousands of modules available.

Q2. Write the syntax and example of if...elif...else statement.

Ans: Syntax: if condition1: statement(s) elif condition2: statement(s) else: statement(s) Example: n=85; if n>=90: print('A+') elif n>=80: print('A') else: print('B')

Q3. Differentiate between for loop and while loop.

Ans: for loop: Used when number of iterations is KNOWN. Syntax: for i in range(n): statement. while loop: Used when number of iterations is UNKNOWN. Syntax: while condition: statement. The for loop is safer as it rarely causes infinite loops; while loop needs manual counter update.

Q4. Explain built-in functions and user-defined functions with examples.

Ans: Built-in functions are already defined in Python: print(), input(), len(), max(), min(), type(), range(). They can be used directly without definition. User-defined functions are created by the programmer using def keyword: def greet(name): print('Hello,', name) greet('Ram') # Output: Hello, Ram

Q5. Write a program to find the sum and average of 5 numbers entered by the user.

Ans: total = 0 for i in range(5): n = float(input('Enter number: ')) total += n average = total / 5 print('Sum =', total) print('Average =', average)

Q6. Explain list with its methods. Give examples of at least 5 methods.

Ans: A list is an ordered, mutable (changeable) collection of items enclosed in [ ]. Methods: append(x) — adds x at end; remove(x) — removes first x; pop(i) — removes at index i; sort() — sorts ascending; reverse() — reverses; insert(i,x) — inserts x at index i; index(x) — returns index of x; count(x) — counts x; len() — returns length.

Q7. What is a dictionary? How is it different from a list?

Ans: A dictionary is a collection of key-value pairs enclosed in { }. Example: d = {'name':'Ram', 'age':16}. Difference: Lists use integer index (0,1,2...) to access elements; Dictionaries use custom keys. Lists maintain order of insertion; Dictionary keys must be unique. Lists use [ ]; Dictionaries use { }.

Q8. Explain local and global variables with examples.

Ans: Local Variable: Defined inside a function. Accessible only within that function. Example: def show(): x = 10 # local print(x) Global Variable: Defined outside all functions. Accessible anywhere in the program. Example: y = 20 # global def show(): print(y) # accessing global

C. LONG ANSWER QUESTIONS (5–8 marks)

Q1. Write a Python program to display multiplication table of a number entered by the user. Also explain each line.

Ans: n = int(input('Enter a number: ')) for i in range(1, 11): print(n, 'x', i, '=', n*i) Explanation: Line 1: Takes integer input from user. Line 2: Loop runs from 1 to 10 (range(1,11)). Line 3: Prints multiplication: n × i = result.

Q2. Write a Python program to check whether a number is positive, negative, or zero using if...elif...else.

Ans: n = float(input('Enter a number: ')) if n > 0: print(n, 'is Positive') elif n < 0: print(n, 'is Negative') else: print('The number is Zero')

Q3. Write a program to find the largest number in a list.

Ans: numbers = [] for i in range(5): n = int(input('Enter number: ')) numbers.append(n) largest = numbers[0] for x in numbers: if x > largest: largest = x print('Largest number:', largest) # Alternative: print(max(numbers))

Q4. Write a Python program using a user-defined function to check if a number is even or odd.

Ans: def check_even_odd(n): if n % 2 == 0: return 'Even' else: return 'Odd' num = int(input('Enter a number: ')) result = check_even_odd(num) print(num, 'is', result)

Q5. Explain different types of errors in Python with examples.

Ans: 1. Syntax Error: Occurs when Python grammar rules are violated. Example: if x = 5: (should be ==). Python shows error before running. 2. Runtime Error: Occurs during execution. Example: x = 10/0 causes ZeroDivisionError. Program crashes. 3. Logical Error: Program runs without error but gives wrong output. Example: Average = total * n instead of total / n. Hardest to detect.

Q6. Write a Python program to count vowels in a string entered by the user.

Ans: s = input('Enter a string: ') count = 0 for ch in s.lower(): if ch in 'aeiou': count += 1 print('Number of vowels:', count)

SECTION 10: MULTIPLE CHOICE QUESTIONS (MCQs)

Q1. Who created Python?

a) Dennis Ritchie

b) Guido van Rossum

c) James Gosling

d) Bjarne Stroustrup

✔ **Answer: b) Guido van Rossum**

Q2. Python is which type of language?

a) Low-level

b) Machine-level

c) High-level

d) Assembly

✔ **Answer: c) High-level**

Q3. What is the output of print(10 // 3)?

a) 3.33

b) 3

c) 4

d) 1

✔ **Answer: b) 3**

Q4. What is the output of print(10 % 3)?

a) 3

b) 1

c) 0

d) 3.33

✔ **Answer: b) 1**

Q5. Which function is used to take input from the user?

a) print()

b) read()

c) input()

d) get()

✔ **Answer: c) input()**

Q6. What does input() always return?

a) Integer

b) Float

c) String

d) Boolean

✔ **Answer: c) String**

Q7. What is the index of the first element in a list?

a) 1

b) -1

c) 0

d) None

✔ **Answer: c) 0**

Q8. Which of the following is immutable?

a) List

b) Dictionary

c) Tuple

d) Set

✔ **Answer: c) Tuple**

Q9. What keyword is used to define a function?

a) function

b) define

c) func

d) def

✔ **Answer: d) def**

Q10. What is the output of range(1, 5)?

a) 1,2,3,4,5

b) 1,2,3,4

c) 0,1,2,3,4

d) 1,2,3,4,5,6

✔ **Answer: b) 1,2,3,4**

Q11. Which symbol is used for comments in Python?

a) //

b) /* */

c) --

d) #

✔ **Answer: d) #**

Q12. What is the output of print(2 ** 4)?

a) 8

b) 16

c) 6

d) 24

✔ **Answer: b) 16**

Q13. Which statement exits a loop immediately?

a) exit

b) stop

c) break

d) end

✔ **Answer: c) break**

Q14. What does len(['a','b','c']) return?

a) 2

b) 4

c) 3

d) 0

✔ **Answer: c) 3**

Q15. Python is case-sensitive. True or False?

a) True

b) False

c) Sometimes

d) Depends on version

✔ **Answer: a) True**

Q16. Which method adds an element at the end of a list?

a) add()

b) insert()

c) append()

d) push()

✔ **Answer: c) append()**

Q17. What is the extension of a Python file?

a) .pt

b) .pyt

c) .py

d) .python

✔ **Answer: c) .py**

Q18. Which operator is used for string concatenation?

a) &

b) +

c) *

d) ,

✔ **Answer: b) +**

Q19. What does 'Hello' * 3 produce?

a) HelloHelloHello

b) Hello3

c) Error

d) Hello Hello Hello

✔ **Answer: a) HelloHelloHello**

Q20. Which is NOT a Python data type?

a) int

b) char

c) float

d) str

✔ **Answer: b) char**

Q21. A variable defined inside a function is called?

a) Global variable

b) Local variable

c) Static variable

d) Public variable

✔ **Answer: b) Local variable**

Q22. Which loop is used when the number of iterations is unknown?

a) for loop

b) while loop

c) do-while loop

d) repeat loop

✔ **Answer: b) while loop**

Q23. What does str(25) return?

a) 25

b) 25.0

c) '25'

d) Error

✔ **Answer: c) '25'**

Q24. Which of these creates a dictionary?

a) d = [1,2,3]

b) d = (1,2,3)

c) d = {1,2,3}

d) d = {'a':1, 'b':2}

✔ **Answer: d) d = {'a':1, 'b':2}**

Q25. What is the output of print('Nepal'[0])?

a) N

b) a

c) l

d) e

✔ **Answer: a) N**

SECTION 11: TRUE / FALSE QUESTIONS

Q#	Statement	Answer	Reason
1	Python is a low-level programming language.	False	Python is a HIGH-LEVEL programming language.
2	Python was created by Guido van Rossum.	True	He created Python in 1991.
3	The input() function returns an integer by default.	False	input() always returns a STRING.
4	List is mutable (can be changed) in Python.	True	Lists can be modified after creation.
5	Tuple is mutable in Python.	False	Tuples are IMMUTABLE — they cannot be changed.
6	Python is case-sensitive.	True	name and Name are different variables.
7	range(1,5) generates: 1, 2, 3, 4, 5.	False	range(1,5) generates 1, 2, 3, 4 (stop not included).
8	The break statement skips to the next iteration of a loop.	False	break EXITS the loop. continue skips to next iteration.
9	The def keyword is used to define a function.	True	def function_name(): is the syntax.
10	A local variable is accessible outside the function.	False	Local variables are accessible ONLY inside the function.
11	// operator in Python performs floor division.	True	10 // 3 = 3 (integer quotient).
12	Dictionary keys can be duplicate.	False	Dictionary keys must be UNIQUE.
13	Python uses curly braces {} to define code blocks.	False	Python uses INDENTATION, not curly braces.
14	The ** operator is used for exponentiation in Python.	True	2 ** 3 = 8.
15	A while loop is preferred when the number of iterations is known.	False	for loop is used when iterations are KNOWN; while loop wh

SECTION 12: FILL IN THE BLANKS

Q#	Statement	Answer
1	Python was created by _____ in the year _____.	**Guido van Rossum, 1991**
2	Python is a _____ level programming language.	**high**
3	The _____ function is used to display output in Python.	**print()**
4	The _____ function is used to take input from the user.	**input()**
5	input() function always returns a _____ data type.	**string (str)**
6	In Python, indentation uses _____ to define a block of code.	**spaces or tabs**
7	The _____ keyword is used to define a function in Python.	**def**
8	A _____ is an ordered, mutable collection using square brackets.	**list**
9	A _____ is an ordered, immutable collection using parentheses.	**tuple**
10	A _____ stores data as key-value pairs using curly braces.	**dictionary**
11	The _____ statement immediately exits a loop.	**break**
12	The _____ statement skips the current iteration of a loop.	**continue**
13	range(1, 10) generates numbers from 1 to _____.	**9**
14	The _____ operator is used for exponentiation in Python.	**** (double asterisk)**
15	The _____ operator returns the remainder of division.	**% (modulus)**
16	A variable defined inside a function is called a _____ variable.	**local**
17	A variable defined outside all functions is called a _____ variable.	**global**
18	The _____ method adds an element at the end of a list.	**append()**
19	Python file is saved with _____ extension.	**.py**
20	The _____ function returns the number of elements in a list or string.	**len()**

SECTION 13: ONE-MARK QUESTIONS

Q#	Question	Answer
1	What does Python stand for?	Python is not an acronym. It is named after the British comedy show 'Mo
2	Name the creator of Python.	Guido van Rossum
3	In which year was Python created?	1991
4	Name two built-in functions in Python.	print() and input()
5	What is the default return type of input() function?	String (str)
6	What symbol is used for single-line comments in Python?	# (hash symbol)
7	Write the syntax of a function definition.	def function_name(parameters):
8	What does // operator do in Python?	Floor division (returns integer quotient). Example: 10//3 = 3
9	What does ** operator do?	Exponentiation (power). Example: 2**3 = 8
10	What does % operator return?	The remainder of division. Example: 10%3 = 1
11	What is the index of the last element in a list of 5 items?	Index 4 (or -1 using negative indexing)
12	Name one method to add an element to a list.	append()
13	Name one method to remove an element from a list by value.	remove()
14	What is the difference between a list and a tuple?	List is mutable; Tuple is immutable.
15	Write any one string method.	upper(), lower(), strip(), replace(), split(), find() — any one
16	What does len('Python') return?	6
17	What is the output of 'Ha' * 3?	HaHaHa
18	What is a syntax error?	An error caused by incorrect Python grammar/syntax.
19	What is an infinite loop?	A loop that never terminates because its condition always stays True.
20	Write one keyword used in Python.	if, else, elif, for, while, def, return, break, continue, in, and, or, not — any

SECTION 14: QUICK REVISION SHEET

■ KEY DEFINITIONS TO MEMORIZE

- Python: High-level, interpreted, general-purpose language by Guido van Rossum (1991).
- Variable: Named memory location to store data.
- print(): Displays output on screen.
- input(): Takes user input as a string.
- if/elif/else: Decision-making statements.
- for loop: Repeats code for each item in sequence.
- while loop: Repeats code while condition is True.
- break: Exit loop. | continue: Skip iteration.
- def: Defines a function. | return: Returns value from function.
- List [ ]: Mutable, ordered collection.
- Tuple ( ): Immutable, ordered collection.
- Dictionary { }: Key-value pairs.
- Parameter: Variable in definition. | Argument: Value in call.
- Local variable: Inside function. | Global variable: Outside function.
- Indentation: Spaces/tabs that define code blocks in Python.

■ SYNTAX QUICK REFERENCE

```
# Output          print(value)
# Input           x = input('msg')  →  x = int(input('msg'))
# if              if cond: ... elif cond: ... else: ...
# for             for i in range(start, stop, step):
# while           while condition:  (update counter!)
# function        def name(param):  ...  return value
# list            a=[1,2,3]  a.append(x)  a.remove(x)  a.sort()
# dict            d={'k':v}  d['k']  d.keys()  d.values()
# string          s.upper() s.lower() s.replace() s.split() s[a:b]
# type convert    int(x)  float(x)  str(x)  bool(x)
# operators       + - * / // % **  == != > < >= <=
# logical         and  or  not
```

■ IMPORTANT RULES

- Python is CASE-SENSITIVE.
- Blocks are defined by INDENTATION (no {} like C/Java).
- input() always returns STRING — convert with int() or float().
- range(start,stop,step): stop value is NOT included.
- List index starts from 0. Negative index: -1 is last element.
- Tuple cannot be modified after creation.

- Dictionary keys must be unique.
- while loop MUST have counter update to avoid infinite loop.
- def and return are the two most important function keywords.
- Comments start with # and are ignored by Python.

■ COMMON ERRORS SUMMARY

■ COMMON EXAM MISTAKES

- **Syntax Error: Missing colon (:) after if/for/while/def.**
- **Runtime Error: ZeroDivisionError, IndexError, TypeError.**
- **Logical Error: Wrong formula/condition — program gives wrong output.**
- **TypeError: Using input() result directly in math without int() conversion.**
- **IndentationError: Mixed tabs and spaces.**
- **NameError: Using variable before assigning a value.**

SECTION 15: EXAM TIPS & MEMORY TRICKS

■ Memory Tricks

■ **Remember "G.V.R. 1991":** Guido Van Rossum created Python in 1991.

■ **SOFT = String Output Float/int Time:** input() takes String, Output is with print, convert to Float or Int, Takes time (waits).

■ **"LiT" for Mutability:** List = True (mutable). Tuple = False (immutable). L before T in alphabet.

■ **PEMDAS for operators:** Parentheses → Exponent(**) → Multiply(*) → Divide(/) → Add(+) → Subtract(-).

■ **"BCA" for loops:** Break = Completely stops. Continue = Advances. (B before C in alphabet = Break first).

■ **'def + return' = Complete function:** Always need def to START and return to GIVE BACK a value.

■ **"KV Store" for Dictionary:** Dictionary stores Key-Value pairs like a real dictionary: word → meaning.

■ **range(1,11) trick:** Add 1 to your last number! If you want 1 to 10, write range(1, 11).

■ **For = Fixed, While = Whenever:** For loop = Fixed iterations. While loop = Whenever condition is met.

■ Exam Day Tips

■ IMPORTANT FOR SEE EXAM

- ALWAYS write proper indentation in Python programs. Use 4 spaces consistently.
- ALWAYS write the colon (:) after if, else, elif, for, while, def.
- When writing programs, trace through your code mentally to check output.
- For input questions, ALWAYS show what int() or float() conversion is needed.
- In difference tables, use at least 4 points of difference for full marks.
- Write program output clearly and underline it for visibility.
- For function programs, always show both the definition AND the function call.
- In definition questions, start with 'Python is...' — don't just give the feature list.
- For loop programs: double-check range() values — the stop value is NOT included.
- In while loop programs: ALWAYS write the counter update line (i += 1) — examiners check this.
- When asked for 'syntax', write it exactly with proper placeholder names like function_name, condition.
- Revise ALL difference tables — they appear every year in SEE exams.
- MCQs: Eliminate wrong answers first, then pick from remaining options.
- If unsure of a definition, write what you know confidently — partial marks are awarded.
- Practice writing at least 5 programs by hand before the exam.

■ BEST OF LUCK FOR YOUR SEE EXAMINATION! ■■

Study hard, practice programs, and you will succeed!